

CSE 318: Artificial Intelligence Sessional

Online 01

B1

N-Puzzle on a Torus Board

Problem Statement

Modify your existing N-Puzzle solver so that the board behaves like a **torus**. If the blank moves out of one edge of the board, it should appear from the opposite edge.

For example, moving left from the first column takes the blank to the last column of the same row. Similarly, moving up from the first row takes it to the last row of the same column.

You may assume that all input boards are solvable. Do not use the old inversion-parity solvability check.

Additional Rule: Wraparound Move Cost

Each normal move costs 1.

Each wraparound move costs 2.

So your program should print the **minimum total cost**, not simply the minimum number of moves.

Heuristic Function

Use **toroidal Manhattan distance** as the heuristic.

Let k be the board size. For a $k \times k$ board, each tile position is represented by its row and column.

For a tile t , let its current position be (r_t, c_t) and its goal position be (r_t^*, c_t^*) .

$$\Delta r_t = |r_t - r_t^*|, \quad \Delta c_t = |c_t - c_t^*|$$

$$d_r(t) = \min(\Delta r_t, k - \Delta r_t)$$

$$d_c(t) = \min(\Delta c_t, k - \Delta c_t)$$

Therefore,

$$h(n) = \sum_{\text{tiles } t} (d_r(t) + d_c(t))$$

The blank tile 0 should not be included in the heuristic calculation.

Output

Print the minimum total cost and the complete board sequence from the initial board to the goal board.

Here, the cost printed with each step should represent the accumulated path cost from the initial board to that board configuration.